

ESE 499 Undergraduate Research –Summer 2026

Linux Installation

**Comparative Analysis of Environment-Aware Resilient Wireless
Communication Links in Disaster Scenarios Using SDR**

Name: Sohee Kim

Student ID: 117643907

Date: June 11, 2026

Email: sohee.kim.3@stonybrook.edu

Advisor: Prof. Harbans Dhadwal



1. Introduction & Motivation

This technical report details the successful implementation of a native, non-virtualized **Dual-Boot Environment** featuring **Windows 11** and **Ubuntu LTS Linux** running on a single non-volatile storage drive (SSD).

Unlike Type-2 Hypervisors (Virtual Machines) or compatibility layers (WSL), this native hardware-level deployment ensures that the Linux operating system has direct access to the physical CPU cores, system RAM, and hardware peripherals (e.g., Software Defined Radio hardware, GPU acceleration). This layout guarantees zero-latency execution required for continuous Digital Signal Processing (DSP) and complex scientific computations.

2. System Architecture & Prerequisites

The installation was engineered around modern firmware standards, utilizing the Unified Extensible Firmware Interface (UEFI) and the GUID Partition Table (GPT) partitioning scheme to host two distinct file systems securely.

- Hard Drive Partition Allocation Scheme

Storage Medium: NVMe M.2 Solid State Drive (SSD)

- Partition 1 (System Boot): EFI System Partition (ESP) — FAT32 Format [Contains both Windows Boot Manager and GNU GRUB binary loaders].
- Partition 2 (Host OS): Windows Main Partition — NTFS Format.
- Partition 3 (Linux Root Directory /): Ubuntu Linux OS and Core Binaries — ext4 Journaling File System.
- Partition 4 (Virtual Memory): Linux Swap Area — Utilized as low-priority paging file when hardware memory thresholds are reached.

3. Step-by-Step Installation & Configuration Procedure

Phase 1: Pre-Installation & Windows Partition Shrinking

To allocate unallocated physical space for the secondary operating system without compromising existing Windows data:

1. Launched the native Windows Disk Management utility (diskmgmt.msc).
2. Located the primary NTFS volume (C: drive).
3. Right-clicked the volume and selected "Shrink Volume...".
4. Calculated and reduced the partition boundary by [Specify Allocated Size, e.g., 64000 MB / 64 GB] to generate a continuous block of Unallocated Space.



Phase 2: Live USB Installation Media Provisioning

1. Acquired the official Ubuntu LTS Desktop ISO image file.

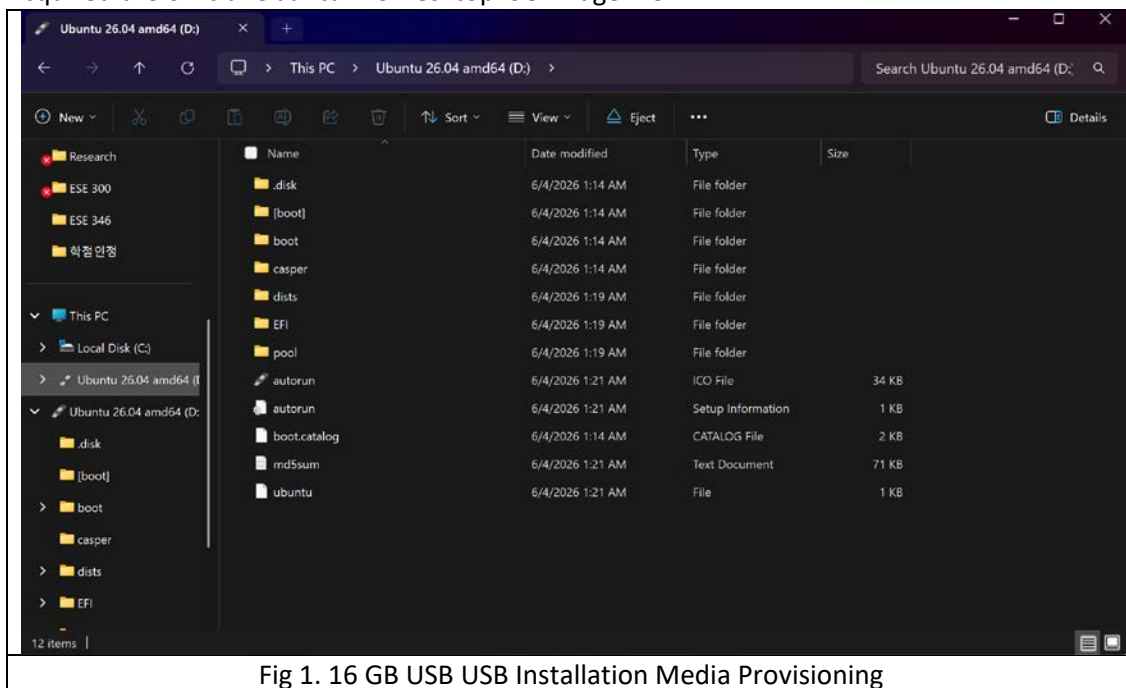


Fig 1. 16 GB USB USB Installation Media Provisioning

2. Formatted a secondary 16 GB USB flash drive into a live installation medium using the open-source utility Rufus.
3. Targeted the GPT partition scheme and forced the Target System to UEFI (non-CSM) to prevent legacy MBR conflicts during the physical boot phase.

Phase 3: Firmware Configuration & Boot Intervention

1. Initiated a hardware reboot sequence, repeatedly asserting the system function key (F12 / F2) to enter the motherboard UEFI Firmware Settings.
2. Verified that Intel Virtualization Technology (VT-x) or AMD equivalent was activated.
3. Modified the boot order array to prioritize the UEFI USB Boot Device over the default Internal Windows Boot Manager.
4. Saved parameters and executed a cold boot into the GNU GRUB live menu.

Phase 4: Manual Partitioning & Linux Installation

Upon booting into the graphical installer interface, selected "Try or Install Ubuntu" -> "Something else"(Advanced Manual Partitioning Mode). This explicit manual routing prevented the automated installer from clearing the existing Windows partition.



The unallocated space generated in Phase 1 was subdivided as follows:

Partition Type	File System	Mount Point	Description
Root Partition	ext4	/	Stores the Linux kernel, system configurations (/etc), and user spaces (/home).
Swap Space	swap	N/A	Virtual memory mapping area to support stable multitasking and memory dump management.

Phase 5: Account Provisioning and Final Initialization

1. Configured local timezone geographical parameters.
2. Defined the user identity infrastructure (Username: sohee, assigned network hostname, and established security password hashes).
3. Monitored the automated extraction of core binaries and final GRUB boot configurations.
4. Upon receiving the system notification *"Installation is Complete"*, removed the physical USB flash drive from the port and executed a hard reset.

4. Post-Installation Multi-Boot Verification & Diagnosis

1. GRUB Bootloader Verification



Fig 2. Multi-Boot

upon system initialization, the screen correctly halts at the GNU GRUB (Grand Unified Bootloader) interactive menu, prompting the engineer to explicitly route the system to either:

- Ubuntu (Directing the kernel to load from the ext4 partition).

- Windows Boot Manager (Redirecting the master firmware control back to the original Windows environment).

2. Resolution of System Time Desynchronization

Due to conflicting methodologies between how Windows interprets the Hardware Real-Time Clock (RTC) as local time and how Linux interprets it as Coordinated Universal Time (UTC), a time offset initially occurred. This was diagnosed and permanently resolved within the Linux kernel terminal using the following synchronization instruction:

```
timedatectl set-local-rtc 1 --adjust-system-clock
```

3. Environment Stability Diagnostics

The integrity of the file systems and core kernel specifications were verified post-deployment via standard command-line tools:

```
# Verify kernel release version and hardware architecture
uname -r

# Verify distribution configuration parameters
lsb_release -a

# Map physical volume mount paths and filesystem data storage
df -h
```

5. Conclusion

The native dual-boot deployment is fully operational. The system stably switches between environments via the GRUB interface, providing an optimized, bare-metal platform for advanced electrical engineering software development and SDR experimentation without operational overhead or structural conflicts.